Technical Take-Home Assessment: Wealth Portfolio Tracker

Role Scope	Full-Stack Software Engineer
Deadline	**Sunday 25 October 2026**, end of the day (Damascus time)
Submission	Reply to the email you received with the repository link and the video link

01 Overview

In this assessment, you will design and build a prototype for a **Wealth Portfolio Tracker**—a tool that allows users or households to manage multi-asset portfolios (stocks, crypto, real estate, cash) with access permissions and dynamic updates.

Because AI agents handle boilerplate generation instantly, this task is **not** a speed-typing test. It evaluates your **architectural reasoning, feature prioritization, stack trade-offs, and craftsmanship** when building software in the age of AI-assisted engineering.

What we want to see is that you understand every line of code and every architectural decision, and that everything is well thought out—not AI-generated code you cannot explain.

02 System Capabilities (Select & Scope Intentionally)

You are **not required to implement every feature listed below**. Choose a subset of features that best showcases your core strengths and architectural thinking. Part of the evaluation is observing what you choose to build, what you defer, and why.

- **Data & Permission Modeling:** Multi-asset records, transaction history, and fine-grained authorization policies (e.g., `Owner`, `Editor`, `Viewer` roles).
- **Embedded AI Agent / Assistant:** An in-app agent that can perform actions or answer queries over portfolio data (e.g., portfolio summaries, rebalancing suggestions, or executing transaction logging via tool calling).
- **Async Processing & External Data:** Scheduled market data updates, background valuation sync, or job queue workflows.
- **Real-Time / Multiplayer Features:** Live portfolio value updates across connected clients, collaborative notes, or active user presence.
- **UI/UX & Craftsmanship:** Clean dashboard layout, robust forms, error handling, streaming responses, loading states, and intentional design (avoiding generic “AI slop”).

03 Stack Selection: Proposed Reference vs. Choice

Below is an **optional reference stack** representative of tools we frequently use. **You are free to use these, modify the combination, or pick an entirely different language/ecosystem** (e.g., Go, Python, Elixir, Rust, Ruby):

Domain	Reference Option	Notes
Frontend / App	Next.js / React	Full-stack framework or standalone SPA.
Backend / API	Express, Next.js API Routes / Actions	Separate API tier vs. unified server layer.
Data & Access	ZenStack, Prisma, Drizzle	Schema declaration, ORM, or integrated access rules.
Embedded AI Agent	Vercel AI SDK, LangChain/LangGraph, Raw LLM API	Framework choice, tool calling loop, and streaming integration.
Async Jobs	BullMQ, Redis	Background workers or queue managers.
Realtime	Liveblocks, PartyKit, WebSockets, SSE	State sync or real-time event streaming.
UI System	Tailwind CSS, Shadcn UI	Design primitives and styling.

*If you choose a non-TypeScript stack (or a decoupled setup), be prepared to explain how you guarantee **end-to-end type safety** or contract integrity across system boundaries.*

04 Deliverables

1. A Plan (the minimum deliverable)

A written plan that shows how deeply you understand the requirements and how far you went into planning each component. A plan alone is an acceptable submission; a fully working product is the final goal, but it is not the point.

2. Source Code Repository

A clean, runnable Git repository with clear setup instructions in the `README.md`, as far as you got.

3. Video Walkthrough

There is **no written report required**. Instead, record a video (10 to 15 minutes, a screen recording with your voice) that walks us through what you have done, covering:

- **Scope & Feature Prioritization:** What features did you choose to build, what did you skip, and why?
- **Agent Architecture & Framework Selection:** Why did you pick your specific AI framework/SDK? What tools did you expose to the agent, and how did you structure tool execution and streaming?
- **Permission & Security Boundaries:** How do permissions hold up when an agent accesses data or mutates state? Is the user's role/access policy enforced within the agent's context?
- **Stack Selection & Trade-offs:** Why did you choose or decline specific tools? (e.g., Why keep/drop Express? Why select your specific data layer?)
- **Agentic Execution:** How did you direct external AI tools/coding agents to build this system efficiently?

Share the video as a link (unlisted YouTube, Google Drive, or Loom).

05 What We Evaluate

- **Prioritization & Scope Control:** Making smart trade-offs under time constraints.
- **Architectural Pragmatism:** Choosing the right tool for the job without adding unnecessary framework bloat.
- **Agent Integration Quality:** Clean tool definition, security-aware agent execution, and responsive streaming UX.
- **Security & Modeling:** Clean data structures and explicit, non-bypassable access controls across standard API routes and AI agent actions.
- **AI Orchestration:** Using meta-level AI tools to deliver production-minded code quality rather than unchecked boilerplate.